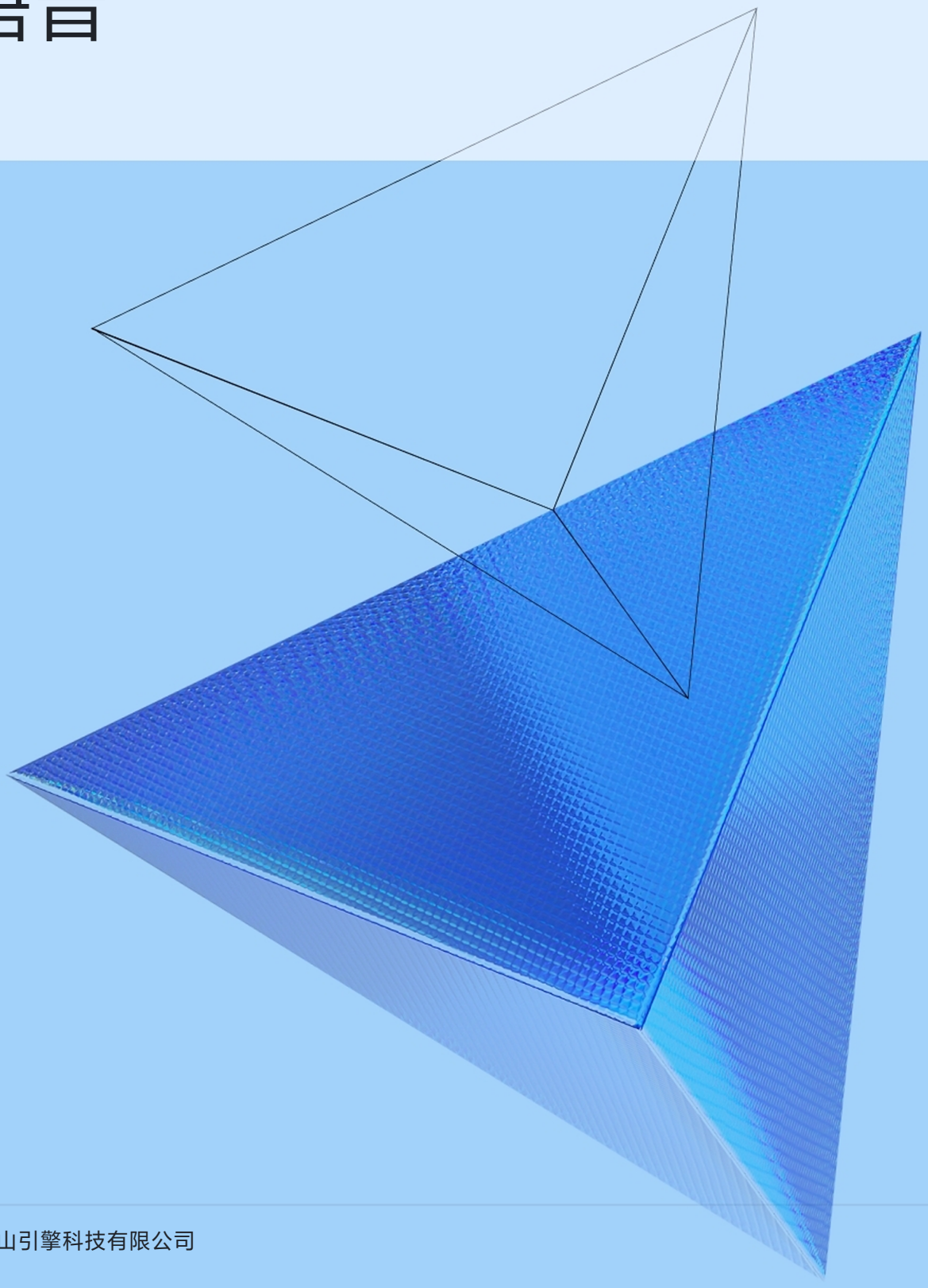
火山引擎

一句话识别

豆包语音



版权所有©北京火山引擎科技有限公司

法律声明

本《豆包语音》的所有内容，包括但不限于文字、商标、架构、图示、图片、页面布局等，其知识产权（著作权、商标权、专利权、商业秘密等）归属于北京火山引擎科技有限公司及其关联公司（火山引擎），非经火山引擎书面同意，任何个人和组织不得复制、使用、修改、转发或以任何违反本《豆包语音》所承载的目的进行传播。

本《豆包语音》陈述内容仅作为产品的通用性介绍和参考性指引，火山引擎保留按“现状”和“当前可用”的形式提供产品和服务的权利。火山引擎不对本《豆包语音》中所载的产品功能、性质、质量、标准等内容进行明示或默示的保证和承诺，最终以您与火山引擎实际签署的协议为准。

如您发现本《豆包语音》有任何错误或歧义，或发现有对本《豆包语音》、产品本身的侵权行为，请与火山引擎取得联系。

联系方式：service@volcengine.com，400-850-0030
（周一至周五 10:00-18:00）

版权所有©北京火山引擎科技有限公司

目录

法律声明----------1
目录----------1
1. 一句话识别----------1

1. 一句话识别

一句话语音识别是指在用户说完一句话后，返回整句识别结果的语音识别方式。相比实时语音识别，准确率更高，适合语音输入法、消息语音转写等无需实时出字、对准确率要求较高的场景

> **说明**
>
> 本文仅介绍 API 参数说明，若需了解接口选型、音频质量要求、交互流程及常见问题，请先阅读《接入必读》文档

> 运行依赖文件

Python

【附件下载】: sauc_python.zip，大小为

Go

【附件下载】: sauc_go.zip，大小为

POST wss://openspeech.bytedance.com/api/v3/sauc/bigmodel_nostream

请求头

**X-Api-Key ** string 必选

API Key 可从 控制台>API Key管理 获取

> **说明**
>
> · 同时支持旧版控制台的鉴权方式，详见旧版控制台鉴权参考示例
> · **API Key 是接入服务的重要凭证，请妥善保管，避免因泄露造成财产损失**

**X-Api-Resource-Id ** string 必选

请求的模型版本，可选值：

- 豆包流式语音识别模型2.0 推荐
  - 小时版：volc.seedasr.sauc.duration
  - 并发版：volc.seedasr.sauc.concurrent
- 豆包流式语音识别模型1.0
  - 小时版：volc.bigasr.sauc.duration
  - 并发版：volc.bigasr.sauc.concurrent

> **说明**
>
> 豆包流式语音识别模型 1.0 为历史版本，新接入用户请优先选择 2.0；现有 1.0 用户可正常使用，不受影响

**X-Api-Request-Id ** string 必选

任务ID，推荐传入随机生成的UUID

请求体

**audio ** dict 必选

**language ** string

指定识别语种，当前支持识别以下语种：

- 中文普通话：zh-CN
- 英语：en-US
- 日语：ja-JP
- 印尼语：id-ID
- 西班牙语：es-MX
- 葡萄牙语：pt-BR
- 德语：de-DE
- 法语：fr-FR
- 韩语：ko-KR
- 菲律宾语：fil-PH
- 马来语：ms-MY
- 泰语：th-TH
- 阿拉伯语：ar-SA
- 意大利语：it-IT
- 孟加拉语：bn-BD
- 希腊语：el-GR
- 荷兰语：nl-NL

版权所有©北京火山引擎科技有限公司

- 俄语：ru-RU
- 土耳其语：tr-TR
- 越南语：vi-VN
- 波兰语：pl-PL
- 罗马尼亚语：ro-RO
- 尼泊尔语：ne-NP
- 乌克兰语：uk-UA
- 粤语：yue-CN

> **说明**
>
> 当 language 参数为空时，模型支持识别以下语种：中文（普通话 / 粤语 / 闽南语 / 四川话 / 陕西话 / 山西话 / 上海话 / 客家话 / 冀鲁官话 / 兰银官话 / 江淮官话）、英语

**format ** string 必选

指定音频格式
可选值：wav / mp3 / ogg / pcm

rate int

指定音频采样率，默认值为 16000

bits int

指定音频采样点位数，默认值为 16

channel int

指定音频声道数，默认值为 1
可选值：
1：mono
2：stereo

request object

model_name string 必选

指定模型名称，目前仅支持 bigmodel

enable_speaker_info bool

版权所有©北京火山引擎科技有限公司

启用说话人分离参数，默认为 `false` ，开启后系统会自动区分音频中的不同说话人，并在识别结果中标注每句话对应的说话人编号

> **说明**
> - 推荐搭配**豆包流式语音识别模型 2.0** 使用
> - 开启后需启用 `show_utterances` 参数，才能获取到说话人分离结果
> - 仅在 `language` 未指定，或指定为 `zh-CN` 时生效
> - 为获得最佳说话人分离效果，建议参考大模型 ASR –说话人分离最佳实践配置相关参数

enable_itn `bool`

启用将语音识别结果转换为规范的书面格式，默认为 `true`
开启后，系统会将语音里口语化的数字、金额及日期等自动转成阿拉伯数字和符号形式，使文本更简洁、更易读
效果示例:

- "一九七零年" → "1970 年"
- "一百二十三美元" → "123 美元"

enable_punc `bool`

启用标点，默认为 `true`
开启后，系统会在识别结果中添加逗号、句号、问号等标点符号，提升文本可读性

enable_ddc `bool`

启用语义顺滑，默认为 `false`
开启后，系统会删除或修正识别结果中的停顿词、语气词、语义重复词等不流畅内容，让文本更连贯、更易读

**output_zh_variant ** `string`

将识别结果输出为繁体中文
可选值:

- `traditional` :简体 → 繁体（大陆）
- `tw` ：简体 → 台湾正体
- `hk` ：简体 → 香港繁体

translate_source_language `string`

指定翻译的源语种，固定为 `yue-CN` ，当前仅支持将粤语识别结果翻译为简体中文

版权所有©北京火山引擎科技有限公司

> **说明**
> 需与 translate_target_language 搭配使用

translate_target_language string

指定翻译的目标语种，固定为 zh-CN ，当前仅支持将粤语识别结果翻译为简体中文

> **说明**
> 需与 translate_source_language 搭配使用

show_utterances bool

启用输出分句、分词、说话人及语音停顿信息，默认为 false

show_speech_rate bool

启用分句信息携带语速，默认为 false
开启后，系统将在分句 additions 中返回语速信息，单位为 token/s

show_volume bool

启用分句信息携带音量，默认 false
开启后，系统将在分句 additions 中返回音量信息，单位为dB

enable_auto_lang bool

启用自动识别语种，默认 false
开启后，系统会自动检测音频所属语种。支持自动识别以下语种：

- 中文普通话 zh-CN
- 英语： en-US
- 日语： ja-JP
- 印尼语： id-ID
- 西班牙语： es-MX
- 葡萄牙语： pt-BR
- 德语： de-DE
- 法语： fr-FR
- 韩语： ko-KR
- 菲律宾语： fil-PH

版权所有©北京火山引擎科技有限公司

- 马来语：`ms-MY`
- 泰语：`th-TH`
- 阿拉伯语 `ar-SA`
- 意大利语 `it-IT`
- 孟加拉语 `bn-BD`
- 希腊语 `el-GR`
- 荷兰语 `nl-NL`
- 俄语 `ru-RU`
- 土耳其语 `tr-TR`
- 越南语 `vi-VN`
- 波兰语 `pl-PL`
- 罗马尼亚语 `ro-RO`
- 尼泊尔语 `ne-NP`
- 乌克兰语 `uk-UA`
- 粤语 `yue-CN`

> **说明**
> 使用该能力时，不支持同时设置 `corpus` 参数，即不支持使用热词、替换词与上下文能力

enable_lid `bool`

启用中英文及方言识别，默认 `false`
支持识别以下语种：中文、英文、上海话、闽南话、四川话、陕西话、粤语
开启后，系统将在 `additions` 中返回语种/场景标签，取值如下：

- `singing_en`：英文唱歌
- `singing_mand`：普通话唱歌
- `singing_dia_cant`：粤语唱歌
- `speech_en`：英文说话
- `speech_mand`：普通话说话
- `speech_dia_nan`：闽南语
- `speech_dia_wuu`：吴语（含上海话）
- `speech_dia_cant`：粤语说话
- `speech_dia_xina`：西南官话（含四川话）
- `speech_dia_zgyu`：中原官话（含陕西话）
- `other_langs`：其它语种（其它语种人声）
- `others`：检测不出（非语义人声和非人声）
- 返回为空则代表无法判断（例如传入音频过短等）

enable_emotion_detection `bool`

版权所有©北京火山引擎科技有限公司

启用情绪检测，默认为 `False`
开启后，系统将在分句 `additions` 中返回对应的情绪标签。支持的情绪标签如下：

- `angry`：表示情绪为生气
- `happy`：表示情绪为开心
- `neutral`：表示情绪为平静或中性
- `sad`：表示情绪为悲伤
- `surprise`：表示情绪为惊讶

enable_gender_detection `bool`

启用性别检测，默认为 `False`
开启后，系统将在分句 `additions` 中返回性别标签（male/female）

enable_age_detection `bool`

启用年龄检测，默认为 `False`
开启后，系统将在分句 `additions` 中返回说话人的年龄（age），返回值为字符串类型的浮点数

> **说明**
>
> 年龄检测是模型基于语音特征对说话人年龄做出的估算，结果仅供参考。实际准确率受录音质量、说话语速、及口音等多种因素影响。

**result_type ** `string`

结果返回方式，默认值为 `full`
可选值：

- `full`：每次识别结果都会重复返回“目前为止的全部文字”
- `single`：增量结果返回，即不返回之前分句的结果

具体差异请参考接入必读文档中的FAQ

vad_segment_duration `int`

指定语义分句的最大静音阈值，默认值为3000，单位为ms。当静音时长超过该值时，识别结果在此处分句

> **说明**
>
> - 此参数仅影响语义分句，不触发判停，不改变 `definite` 返回位置

- 若同时配置了 `end_window_size`，此参数不生效

end_window_size `int`

语音活动检测 (VAD) 的静音判停阈值，单位 ms。当检测到的连续静音时长达到该值时，判定一句话结束并触发分句。默认值为 `800`
取值范围： `[300,5000]`
推荐值： `[800,1000]`

**force_to_speech_time ** `int`

设置音频流起始阶段强制按有声处理的时长，默认值为 `0`，单位为 ms
该参数用于规避音频起始处因静音或弱音导致的过早判停或无法判起问题。设定时长内即使输入为静音也不触发 VAD 判停，超时后恢复正常判停逻辑
推荐值： `1000`

sensitive_words_filter `string`

启用敏感词过滤功能。开启后，可对识别结果中的敏感词做屏蔽或替换处理
示例

```Bash
"sensitive_words_filter":{\"system_reserved_filter\":true,\"filter_with_empty\":[\"敏感词\"],
\"filter_with_signed\":[\"敏感词\"]}"
```

**system_reserved_filter ** `bool`

启用系统内置敏感词库，默认为 `false`，启用后，命中的系统敏感词会被替换为 `*`

**filter_with_empty ** `string`

设置需替换为空字符串的自定义敏感词列表

**filter_with_signed ** `string`

设置需替换为 `*` 的自定义敏感词列表

enable_poi_fc `bool`

启用 POI Function Call，默认为 `false` ，启用后可调用专业的地图领域推荐词服务辅助识别，提高识别准确率
示例：

```SQL
"request": {
  "enable_poi_fc": true,
  "corpus": {
    "context": "{\"loc_info\":{\"city_name\":\"北京市\"}}"
  }
}
```

enable_music_fc `bool`

启用 Music Function Call，默认为 `false` ，开启后，对于语音识别困难的词语，能调用专业的音乐领域推荐词服务辅助识别

corpus `object`

配置语境词典，可自定义配置热词、替换词和上下文信息，配置后可提高特定语境下的词语识别准确率

> **说明**
> - 使用该能力时，不支持同时设置 `enable_auto_lang` 参数
> - 热词总长度不得超过5000词（含 `context` 直传热词与热词表热词）。当热词总长度超过5000词时，系统按热词传入顺序从前向后截断，仅前5000热词生效
> - 热词属于提示性参数，用于引导模型优先识别特定词汇，但并非强制约束。模型会在识别过程中优先考虑热词，但受语音清晰度、语境等多种因素影响，不保证所有热词都能 100% 正确转写
> - 热词的效果和选词、数量、格式都有关，为达到最佳效果建议根据热词与上下文最佳实践 配置热词

**boosting_table_name ** `string`

热词词表名称，可在控制台>自学习平台 配置热词后获取

**boosting_table_id ** `string`

热词词表id，可在控制台>自学习平台 配置热词后获取

版权所有©北京火山引擎科技有限公司

> **说明**
>
> 若传入的 boosting_table_name 和 boosting_table_id 对应的热词词表不一致，则以 boosting_table_id 为准

**correct_table_name ** string

替换词词表名称，可在控制台>自学习平台 配置替换词后获取。配置后，可将模型识别出的特定词汇替换为目标词汇

**correct_table_id ** string

替换词词表id，可在控制台>自学习平台 配置替换词后获取。配置后，可将模型识别出的特定词汇替换为目标词汇

> **说明**
>
> 若传入的 correct_table_name 和 correct_table_id 对应的热词词表不一致，则以 correct_table_id 为准

regex_correct_table_name string

正则替换词表名称，可在控制台>自学习平台 配置正则替换词后获取。相较于替换词的精确匹配替换，正则替换词适合批量格式转换（如日期格式统一、符号标准化）、模糊模式匹配等复杂场景

**regex_correct_table_id ** string

正则替换词表id，可在控制台>自学习平台 配置正则替换词后获取。相较于替换词的精确匹配替换，正则替换词适合批量格式转换（如日期格式统一、符号标准化）、模糊模式匹配等复杂场景

**context ** string

上下文功能。在识别请求中传入辅助上下文信息，帮助模型结合语境提升识别准确率。支持传入热词、对话历史、场景描述等多种类型的上下文信息

> **说明**
>
> - 使用该能力时，不支持同时设置 enable_auto_lang 参数
> - 实际传参时 ** ** **context** ** 需序列化为 JSON 字符串传入**，如： "context": "{\"hotwords\":[{\"word\":\"自定义热词A\"},,\"context_type\":\"dialog_ctx\",\"context_data\":[{\"speaker\":\"bot\",\"text\":\"最近一轮助手的回答\"}"
> - 上下文最大支持上传800 tokens、20轮对话；超出限制时，按时间顺序保留最新内容，截断最早的历史对话

版权所有©北京火山引擎科技有限公司

- 识别效果与热词的选词策略、上下文信息有关，为达到最佳效果建议根据热词与上下文最佳实践配置热词

示例：

Python

```
{
 "hotwords": [
  {"word": "自定义热词A"},
  {"word": "自定义热词B"},
  {"word": "自定义热词C"}
 ],
 "context_type": "dialog_ctx",
 "context_data": [
  {
   "speaker": "user",
   "text": "你能帮我查一下资料吗？"
  },
  {
   "speaker": "bot",
   "text": "当然可以，请问您需要查什么资料？"
  },
  {
   "speaker": "user",
   "text": "帮我查一下最新的汽车资讯。"
  },
  {
   "speaker": "bot",
   "text": "好的，正在为您查找相关的汽车资讯。"
  }
 ]
}
```

hotwords string

热词列表直传，用于提升指定词汇的识别准确率

word string

热词内容

版权所有©北京火山引擎科技有限公司

> **说明**
> · 热词总长度不得超过5000词（含直传热词与热词表热词）。当热词总长度超过5000词时，系统按热词传入顺序从前向后截断，仅前5000热词生效
> · 热词属于提示性参数，用于引导模型优先识别特定词汇，但并非强制约束。模型会在识别过程中优先考虑热词，但受语音清晰度、语境等多种因素影响，不保证所有热词都能 100% 正确转写

context_type string

上下文类型，目前仅支持 dialog_ctx

context_data object

上下文数据列表，用于传入历史对话等语境信息，需同时配置 context_type

**text ** string

历史对话文本，帮助模型理解语境，提升识别准确率

**image_url ** string

图片 URL，用于提供视觉上下文，辅助理解语音内容

> **说明**
> 仅豆包流式语音识别模型 2.0 支持图片输入。当前限制：最多传入 1 张图片，单张大小不超过 500 KB，支持格式为 jpeg / jpg / png

响应

**code ** int

请求状态码。0 表示识别成功，非 0 表示识别失败

event int

会话事件类型标识

**is_last_package ** bool

最后一个响应包标志。 true 表示识别结果已全部返回

**payload_sequence ** int

响应数据包的序号

**payload_size ** int

响应数据 payload 的字节大小

**payload_msg ** object

响应数据主体，包含音频信息与识别结果

**audio_info ** object

音频相关信息

**duration ** int

音频时长，单位为毫秒（ ms ）

**result ** list

识别结果，识别成功后返回

**additions ** object

log_id string

服务端返回的 logid，方便定位问题

**text ** string

音频识别结果文本，识别成功后返回

translated_text string

识别结果的翻译文本。当指定将粤语识别结果翻译为简体中文时，该字段返回翻译后的简体中文内容；未指定翻译时不返回

版权所有©北京火山引擎科技有限公司

**utterances ** list

语音分句信息。满足以下条件时返回：

- 请求参数 show_utterances 设置为 true
- 识别成功

additions string

**fixed_prefix_result ** string

已确定的前缀识别结果

**source ** string

分句结果来源，固定为 stream

**speaker_id ** string

说话人 ID。开启说话人分离 enable_speaker_info=true 后返回

**definite ** bool

当前分句结果是否为最终确定结果。 true 表示该分句不再变化

**end_time ** int

分句结束时间戳（毫秒）

start_time int

分句起始时间戳（毫秒）

text string

分句文本内容

translated_text string

识别结果的翻译文本。当指定将粤语识别结果翻译为简体中文时，该字段返回翻译后的简体中文内容；未指定翻译时不返回

版权所有©北京火山引擎科技有限公司

words list

分词信息列表。请求参数 show_utterances 设置为 true 且识别成功时返回

**start_time ** int

起始时间（毫秒）

**end_time ** int

结束时间（毫秒）

**text ** string

语音文本内容。满足以下条件时返回

- 请求参数 show_utterances 设置为 true
- 识别成功

return (<ApiSample data={"{\"type\":\"multiple\",\"config\":{\"inputName\":\"输入示例\",\"outputName\":\"输出示例\"},\"data\":[{\"title\":\"Python\",\"key\":\"0\",\"inputCode\":\"import asyncio\\nimport json\\nimport protocol\\nimport argparse\\n\\nasync def main():\\n parser = argparse.ArgumentParser(description=\\\"ASR WebSocket Client\\\")\\n parser.add_argument(\\\"--file\\\", type=str,\\n default=\\\"<your_file_path>\\\",\\n help=\\\"Audio file path\\\")\\n parser.add_argument(\\\"--url\\\", type=str, default=\\\"wss://openspeech.bytedance.com/api/v3/sauc/bigmodel_nostream\\\",\\n help=\\\"WebSocket URL\\\")\\n parser.add_argument(\\\"--seg-duration\\\", type=int, default=200,\\n help=\\\"Audio duration(ms) per packet, default:200\\\")\\n parser.add_argument(\\\"--api_key\\\", type=str, default=\\\"<your_api_key>\\\",\\n help=\\\"API key for authorization (new console auth, X-Api-Key)\\\")\\n parser.add_argument(\\\"--app_key\\\", type=str, default=\\\"\\\",\\n help=\\\"App key (legacy console auth, used with access_key)\\\")\\n parser.add_argument(\\\"--access_key\\\", type=str, default=\\\"\\\",\\n help=\\\"Access key for authorization (legacy console auth, used with app_key)\\\")\\n parser.add_argument(\\\"--resource_id\\\", type=str, default=\\\"volc.seedasr.sauc.duration\\\",\\n help=\\\"X-Api-Resource-Id\\\")\\n\\n args = parser.parse_args()\\n if not args.api_key and (not args.app_key or not args.access_key):\\n raise ValueError(\\\"auth required: provide --api_key, or --app_key + --access_key\\\")\\n\\n config = protocol.Config(\\n api_key=args.api_key,\\n app_key=args.app_key,\\n access_key=args.access_key,\\n resource_id=args.resource_id,\\n)\\n payload = {\\n \\\"audio\\\": {\\n \\\"format\\\": \\\"wav\\\",\\n \\\"codec\\\": \\\"raw\\\",\\n \\\"rate\\\": 16000,\\n \\\"bits\\\": 16,\\n \\\"channel\\\": 1\\n },\\n \\\"request\\\": {\\n \\\"model_name\\\": \\\"bigmodel\\\",\\n \\\"enable_itn\\\": True,\\n \\\"enable_punc\\\": True,\\n \\\"enable_ddc\\\":

版权所有©北京火山引擎科技有限公司

True,\\n \\\"show_utterances\\\": True\\n }\\n }\\n async with protocol.AsrWsClient(args.url, args.seg_duration) as client: # 使用async with\\n try:\\n async for response in client.execute(args.file,config,payload):\\n protocol.logger.info(f\\ \"Received response: {json.dumps(response.to_dict(), indent=2, ensure_ascii=False)}\\\")\\n except Exception as e:\\n protocol.logger.error(f\\\"ASR processing failed: {e}\\\")\\n\\nif __name__ == \\\"__main__\\\":\\n asyncio.run(main())\",\"outputCode\":\"{\\n \\\"code\\\": 0,\\n \\ \"event\\\": 0,\\n \\\"is_last_package\\\": true,\\n \\\"payload_sequence\\\": 42,\\n \\\"payload_size\\ \": 2631,\\n \\\"payload_msg\\\": {\\n \\\"audio_info\\\": {\\n \\\"duration\\\": 8040\\n },\\n \\ \"result\\\": {\\n \\\"additions\\\": {\\n \\\"log_id\\\": \\ \"20260810152426979947089E61DF8235A3\\\"\\n },\\n \\\"text\\\": \\\"您好，我是您的 AI 助手。\\\",\\n \\\"utterances\\\": [\\n {\\n \\\"additions\\\": {\\n \\\"fixed_prefix_result\\ \": \\\"\\\",\\n \\\"source\\\": \\\"stream\\\",\\n \\\"speaker_id\\\": \\\"0\\\"\\n },\\n \ \\\"definite\\\": true,\\n \\\"end_time\\\": 2000,\\n \\\"start_time\\\": 280,\\n \\\"text\\\": \ \\\"您好，我是您的 AI 助手。\\\",\\n \\\"words\\\": [\\n { \\\"end_time\\\": 360, \\ \"start_time\\\": 280, \\\"text\\\": \\\"您\\\" },\\n { \\\"end_time\\\": 680, \\\"start_time\\\": 360, \\ \"text\\\": \\\"好\\\" },\\n { \\\"end_time\\\": 880, \\\"start_time\\\": 680, \\\"text\\\": \\\"我\\\" },\ \n { \\\"end_time\\\": 1040, \\\"start_time\\\": 880, \\\"text\\\": \\\"是\\\" },\\n { \\ \"end_time\\\": 1160, \\\"start_time\\\": 1040, \\\"text\\\": \\\"您\\\" },\\n { \\\"end_time\\\": 1320, \\\"start_time\\\": 1160, \\\"text\\\": \\\"的\\\" },\\n { \\\"end_time\\\": 1520, \\\"start_time\\\": 1360, \\\"text\\\": \\\"AI\\\" },\\n { \\\"end_time\\\": 1800, \\\"start_time\\\": 1600, \\\"text\\\": \\ \"助\\\" },\\n { \\\"end_time\\\": 2000, \\\"start_time\\\": 1800, \\\"text\\\": \\\"手\\\" }\\n]\ \n }\\n]\\n }\\n }\\n}\",\"inputLanguage\":\"\",\"outputLanguage\":\"\"},{\"title\":\"Go\",\"key\": \"1\",\"inputCode\":\"package main\\n\\nimport (\\n \\\"context\\\"\\n \\\"flag\\\"\\n \\\"log\\\"\ \n \\\"os\\\"\\n \\\"sync\\\"\\n\\n \\\"byted.org/data-speech/asr-tob-demo/sauc/protocol\\\"\ \n \\\"github.com/bytedance/sonic\\\"\\n)\\n\\nvar (\\n filePath = flag.String(\\\"file\\\", \\ \"<your_file_path>\\\", \\\"audio file path\\\")\\n wsURL = flag.String(\\\"url\\\", \\\"wss://openspeech.bytedance.com/api/v3/sauc/bigmodel_nostream\\\", \\\"request url\\\")\\n segmentDuration = flag.Int(\\\"seg_duration\\\", 200, \\\"audio duration(ms) per packet, default:200\\ \")\\n apiKey = flag.String(\\\"api_key\\\", \\\"<your_api_key>\\\", \\\"API key for authorization (new console auth, X-Api-Key)\\\")\\n appKey = flag.String(\\\"app_key\\\", \\\"\\\", \\\"App key (legacy console auth, used with access_key)\\\")\\n accessKey = flag.String(\\\"access_key\\\", \\ \"\\\", \\\"Access key for authorization (legacy console auth, used with app_key)\\\")\\n resourceID = flag.String(\\\"resource_id\\\", \\\"volc.seedasr.sauc.duration\\\", \\\"X-Api-Resource-Id\\\")\\n)\\n\\nfunc main() {\\n flag.Parse()\\n log.SetFlags(log.Lmicroseconds | log.Lshortfile)\\n\ \n if *apiKey != \\\"\\\" {\\n _ = os.Setenv(\\\"API_KEY\\\", *apiKey)\\n }\\n if *appKey != \\\"\\ \" {\\n _ = os.Setenv(\\\"APP_KEY\\\", *appKey)\\n }\\n if *accessKey != \\\"\\\" {\\n _ = os.Setenv(\\\"ACCESS_KEY\\\", *accessKey)\\n }\\n if *resourceID != \\\"\\\" {\\n _ = os.Setenv(\\ \"RESOURCE_ID\\\", *resourceID)\\n }\\n\\n if protocol.ApiKey() == \\\"\\\" && (protocol.AppKey() == \\\"\\\" || protocol.AccessKey() == \\\"\\\") {\\n log.Fatal(\\\"auth required: provide --api_key, or --app_key + --access_key, or set them in ../config.toml\\\")\\n }\\n // 打开日志文件，如果文件不存在则创建\\n file, err := os.OpenFile(\\\"run.log\\\", os.O_CREATE|os.O_WRONLY|os.O_APPEND|os.O_TRUNC, 0666)\\n if err != nil {\\n log.Fatalf(\\\"Failed to open log file: %v\\\", err)\\n }\\n defer func() {\\n _ = file.Close()\\n }()\\n run()\\n}\\n\\nfunc run() {\\n payload := protocol.AsrRequestPayload{\\n User: protocol.UserMeta{\\n Uid: \\\"demo_uid\\\",\\n },\ \n Audio: protocol.AudioMeta{\\n Format: \\\"wav\\\",\\n Codec: \\\"raw\\\",\\n Rate: 16000,\\n Bits: 16,\\n Channel: 1,\\n },\\n Request: protocol.RequestMeta{\ \n ModelName: \\\"bigmodel\\\",\\n EnableITN: true,\\n EnablePUNC: true,\

\n EnableDDC: true,\\n ShowUtterances: true,\\n },\\n }\\n c := protocol.NewAsrWsClient(*wsURL, *segmentDuration)\\n resChan := make(chan *protocol.AsrResponse)\\n wg := sync.WaitGroup{}\\n wg.Add(1)\\n go func() {\\n for res := range resChan {\\n resStr, _ := sonic.MarshalString(res)\\n log.Println(resStr)\\n }\\n wg.Done()\\n }()\\n\\n err := c.Execute(context.Background(), *filePath, resChan, payload)\\n if err != nil {\\n log.Fatalf(\\\"failed to execute: %v\\\", err)\\n return\\n }\\n wg.Wait()\\n}\\n\", \"outputCode\":\"{\\n \\\"code\\\": 0,\\n \\\"event\\\": 0,\\n \\\"is_last_package\\\": true,\\n \\\"payload_sequence\\\": 42,\\n \\\"payload_size\\\": 2631,\\n \\\"payload_msg\\\": {\\n \\\"audio_info\\\": {\\n \\\"duration\\\": 8040\\n },\\n \\\"result\\\": {\\n \\\"additions\\\": {\\n \\\"log_id\\\": \\\"20260810152426979947089E61DF8235A3\\\"\\n },\\n \\\"text\\\": \\\"您好，我是您的 AI 助手。\\\",\\n \\\"utterances\\\": [\\n {\\n \\\"additions\\\": {\\n \\\"fixed_prefix_result\\\": \\\"\\\",\\n \\\"source\\\": \\\"stream\\\",\\n \\\"speaker_id\\\": \\\"0\\\"\\n },\\n \\\"definite\\\": true,\\n \\\"end_time\\\": 2000,\\n \\\"start_time\\\": 280,\\n \\\"text\\\": \\\"您好，我是您的 AI 助手。\\\",\\n \\\"words\\\": [\\n { \\\"end_time\\\": 360, \\\"start_time\\\": 280, \\\"text\\\": \\\"您\\\" },\\n { \\\"end_time\\\": 680, \\\"start_time\\\": 360, \\\"text\\\": \\\"好\\\" },\\n { \\\"end_time\\\": 880, \\\"start_time\\\": 680, \\\"text\\\": \\\"我\\\" },\\n { \\\"end_time\\\": 1040, \\\"start_time\\\": 880, \\\"text\\\": \\\"是\\\" },\\n { \\\"end_time\\\": 1160, \\\"start_time\\\": 1040, \\\"text\\\": \\\"您\\\" },\\n { \\\"end_time\\\": 1320, \\\"start_time\\\": 1160, \\\"text\\\": \\\"的\\\" },\\n { \\\"end_time\\\": 1520, \\\"start_time\\\": 1360, \\\"text\\\": \\\"AI\\\" },\\n { \\\"end_time\\\": 1800, \\\"start_time\\\": 1600, \\\"text\\\": \\\"助\\\" },\\n { \\\"end_time\\\": 2000, \\\"start_time\\\": 1800, \\\"text\\\": \\\"手\\\" }\\n]\\n }\\n]\\n }\\n}\",\"inputLanguage\":\"\",\"outputLanguage\":\"\"}]}"} />

版权所有©北京火山引擎科技有限公司